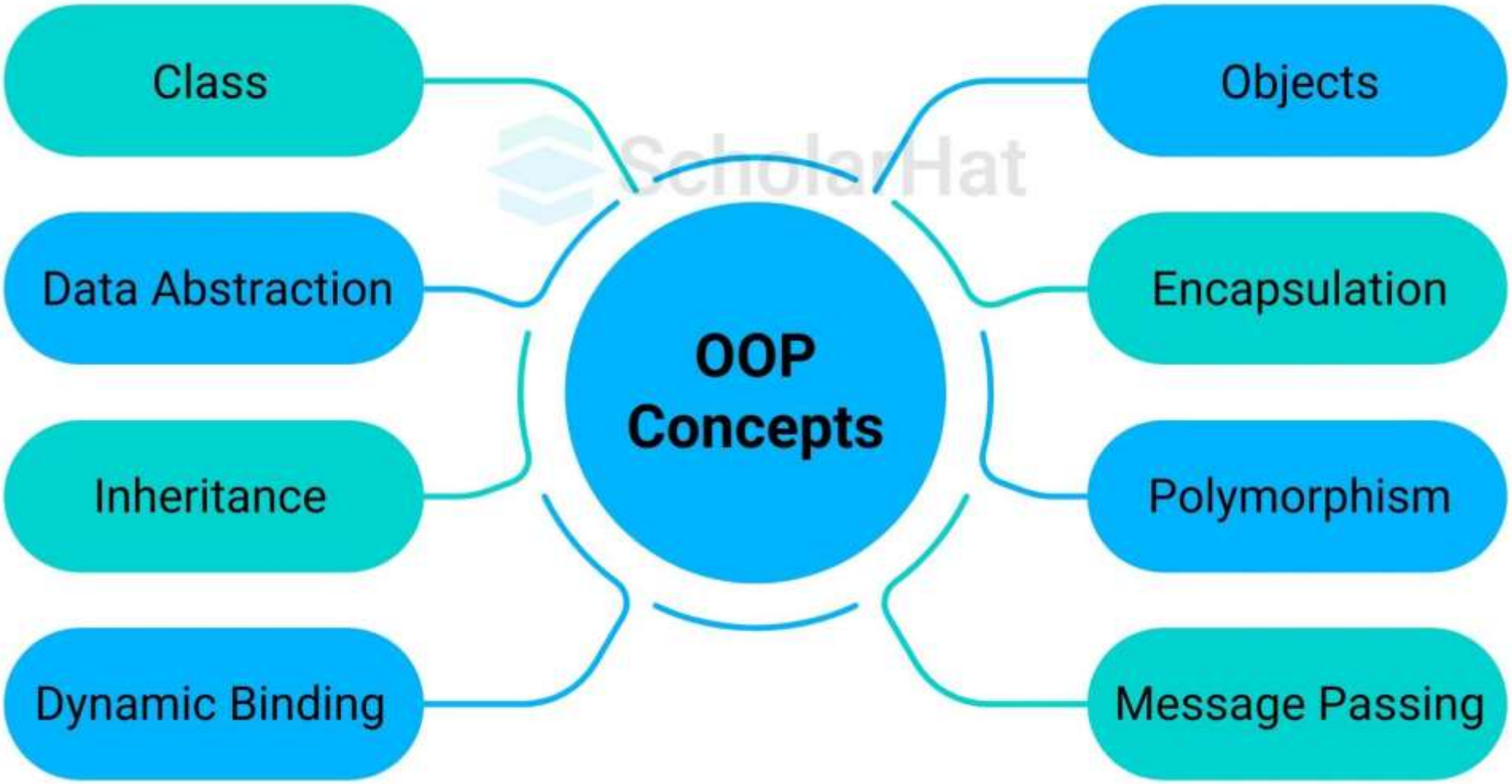
# Module 2: Class and Objects

- **Topics:** Classes as User Defined Data Type, Objects as Instances of Classes, Creating Class and Objects, Creating Objects By Passing Values, Variables & Methods in a Class, Data Abstraction, Data Hiding, Encapsulation, Modularity, Inheritance, Polymorphism
- At the end of Module 2, The students will be able to:
- Understand class, objects and basic concepts of OOPs and implement the same using Python language.

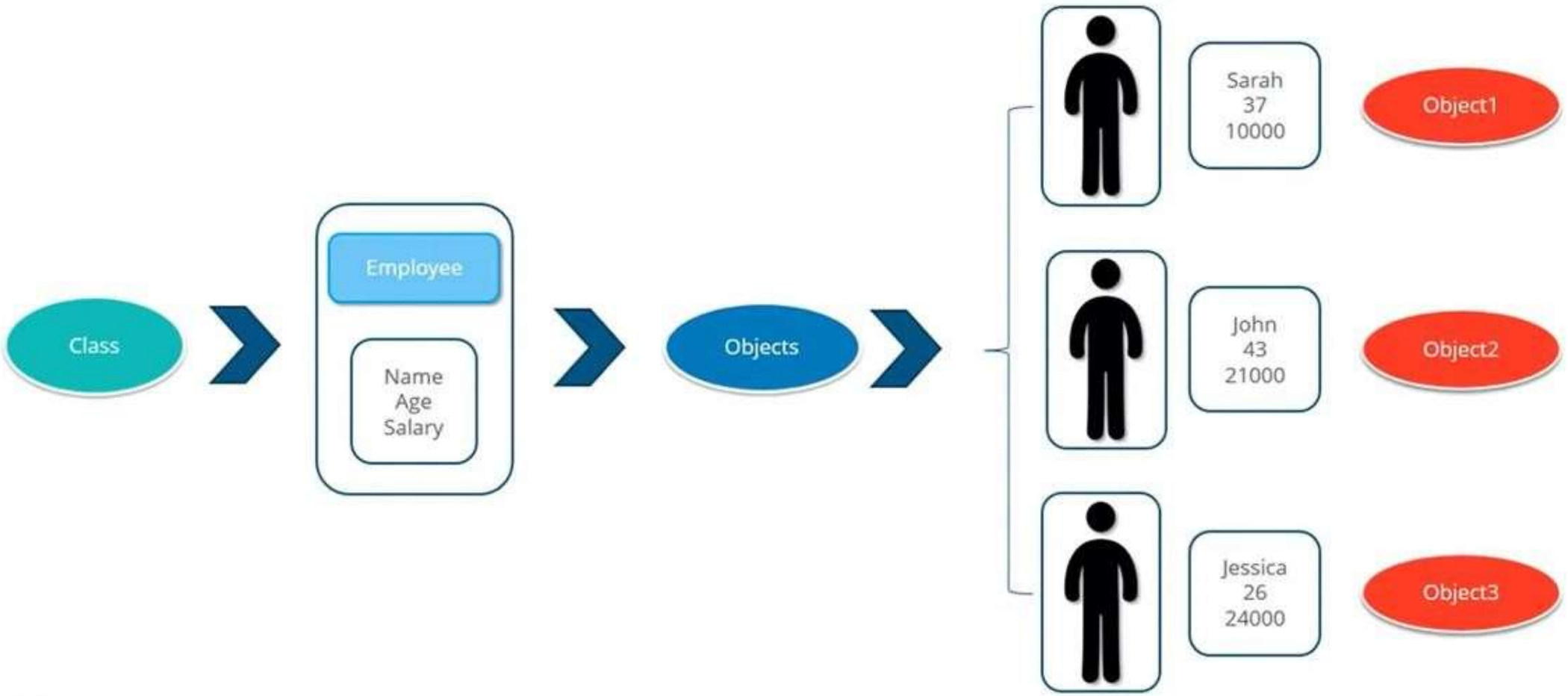
# Session Topics

- Classes as User Defined Data Type, Objects as Instances of Classes, Creating Class and Objects
- Session outcome, The students will be able to:
- Develop programs using class and objects.

# Basic Concepts of OOPs



# Class & Objects



# Class | Object | Reference Variable

**BASICS**:

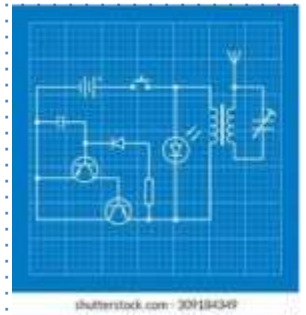
**PLAN** acts as Blueprint (Template) for Building.

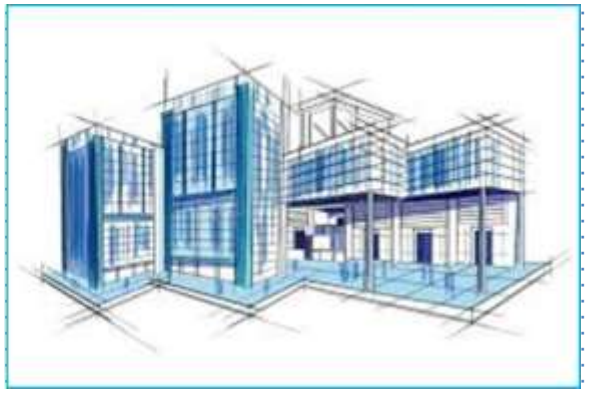
(**PLAN** – as a **CLASS**)

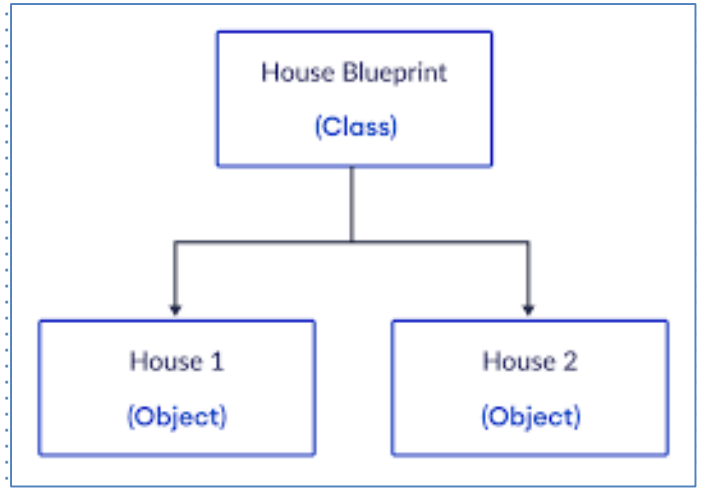
Building constructed based on a plan -> **OBJECTS**

Physical Existence of a class is nothing but **OBJECT**

Once class is Ready ----> **Multiple Objects**



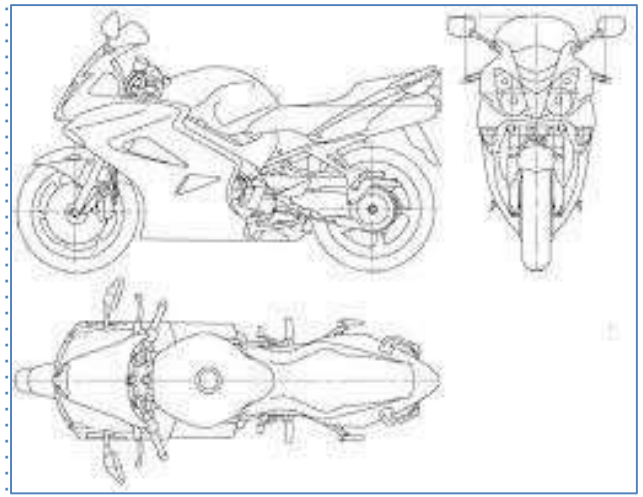




# Example- Class and Objects

- **Design** is a **CLASS**
- Each Physical Bike --->**OBJECT**

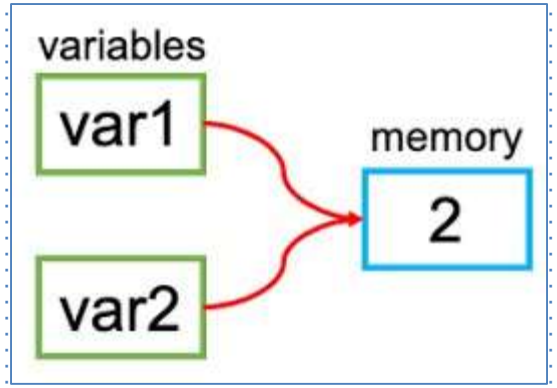
Based on Same design Multiple Bikes manufactured





# Reference Variable

- a variable is not a container that directly stores a value; instead, it acts as a reference or label that points to an object in memory
- When a variable is assigned a value, it is essentially bound to the memory address where that value (an object) resides
- Python uses a pass by **object reference model**



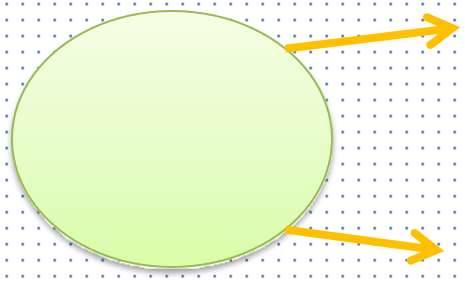
# Class | Object | Reference Variable (Cont..,)

CLASS → OBJECTS -> One to Many

OBJECT → Reference Variable -> one to one

-> one to many

For same OBJECT may offers Multiple References




id(R1)
Id(R2)

# Self Variable

- **Self** is a ***reference variable*** which is always pointing to **current object**. Within the python class to **refer current object** we should use **self variable**.
- The **first argument** to the **constructor** and ***instance method*** should be self
- **PVM** is responsible to provide value for **self argument** and we are **NOT** required to provide ***explicitly***.
- By using **self** we can **declare instance variables.**
- By using **self** we can **access instance variables.**




```
class laptop:
def init (self, brand, ram, os)
      self.ram = ram
      self.os = os
specs = laptop("8GB","Windows")
```

# Constructor

- **Special method in python**
- __init__() at the time of object creation to declare and initialize variables only once
- atleast one argument self python will provide by default
- the arguments can be defined with default values, making them optional to the caller



# Self

- Although you must specify **self** explicitly when defining the method, you don't include it when calling the method.

- Python passes it for you automatically.

**Defining a method:**

```
def set_age(self, num):
    self.age = num
```

**Calling a method:**

```
>>> x.set_age(23)
```

# Method & Constructor

| METHOD | CONSTRUCTOR |
| --- | --- |
| The Name can be Anything | Should be __init__() |
| Method will be executed if we call | Will be executed automatically if we are creating object |
| Per object method can be called any no of times | Per object only once to declare and initialize instance variables |

# Definition of student

```
class student:
    """A class representing a student"""
    def __init__(self, n, a):
        self.full_name = n
        self.age = a
    def get_age(self):
        return self.age
```

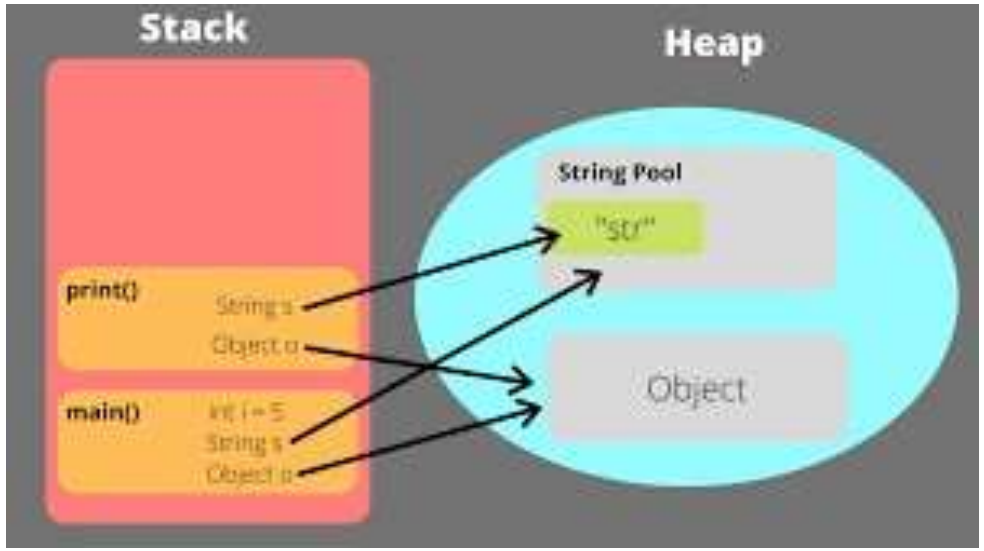
# No Need to “free”

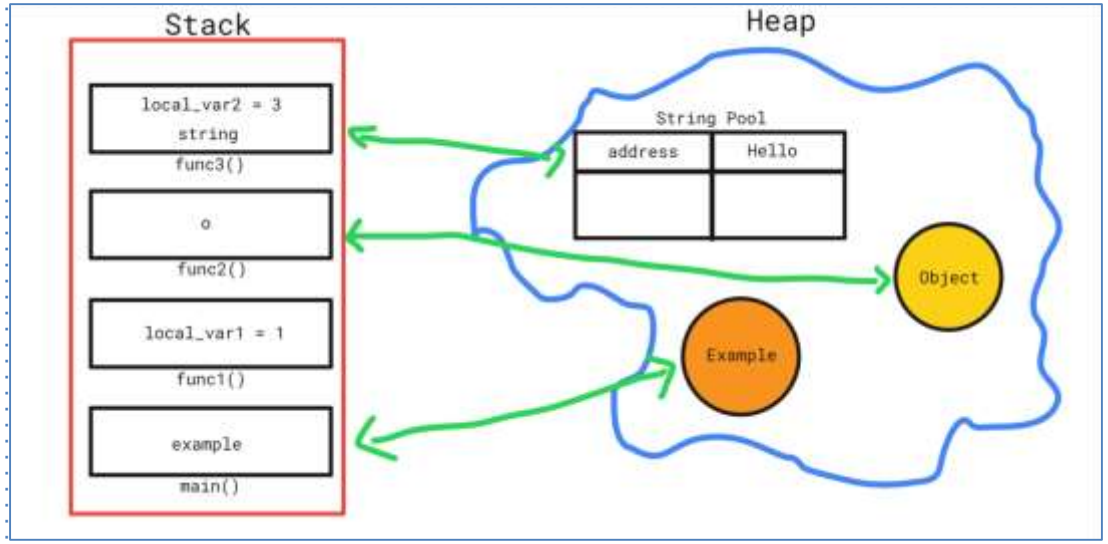
- When you are done with an object, you don’t have to delete or free it explicitly.
  - Python has automatic garbage collection
  - Python will automatically detect when all of the references to a piece of memory have gone out of scope. Automatically frees that memory.
  - There’s no “destructor” method for classes.



# Garbage collection – memory allocation

- **Stack Memory:** Stores local variables and function call information
- **Heap Memory:** Stores all Python objects and dynamic data structures.



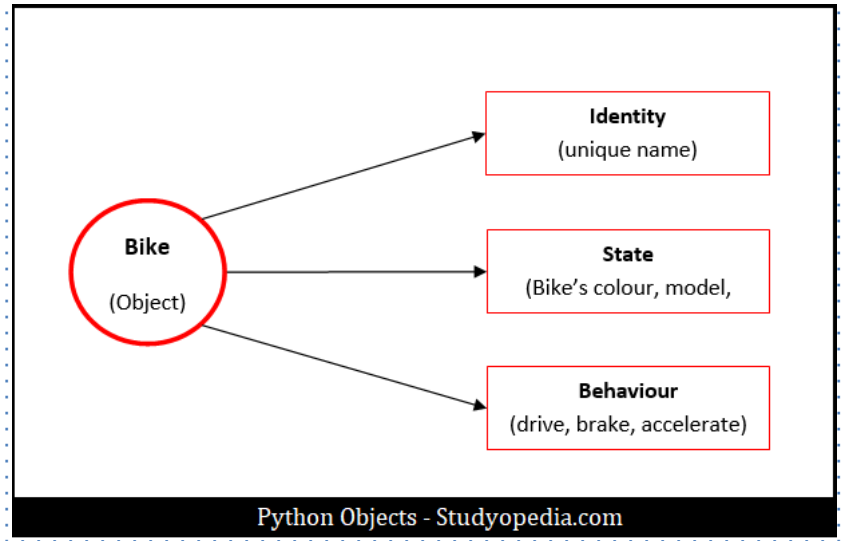
# garbage collection

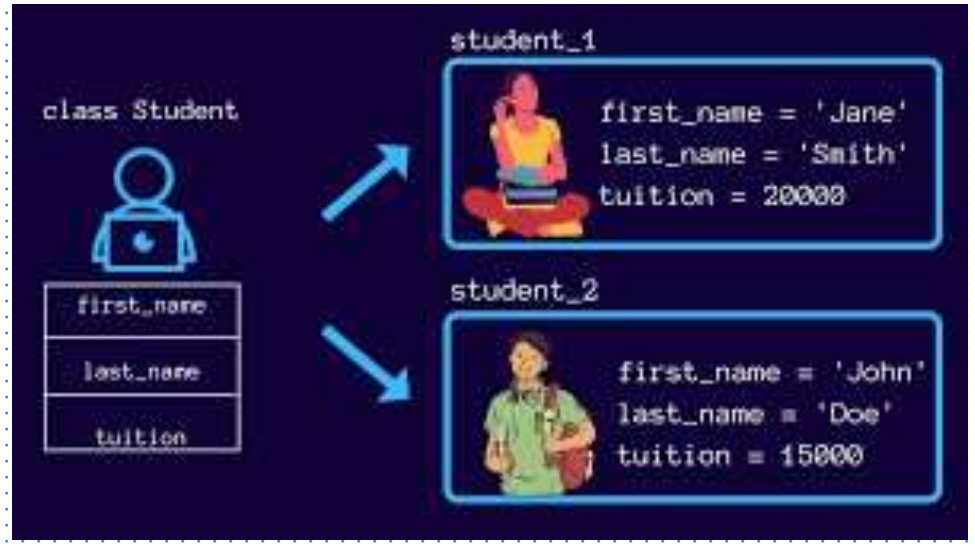
- **Reference Counting:** primary and most frequent form of garbage collection - objects refer to each other, preventing their reference counts from ever reaching zero
- **Generational Garbage Collection (Cyclic References):** Objects are categorized into generations (0, 1, and 2), with newer objects in Generation 0 and older, **surviving objects moving to higher generations.**
- **Free Lists:** Python often uses free lists to manage memory - avoiding repeated system memory requests



# Creating Objects By Passing Values

# Variables & Methods in a Class






**# Sample Program 1**

```
class Room:
    length = 0.0
    breadth = 0.0
    # method to calculate area
    def calculate_area(self):
        print("Area of Room =", self.length * self.breadth)

# create object of Room class
study_room = Room()

# assign values to all the properties
study_room.length = 42.5
study_room.breadth = 30.8

# access method inside class
study_room.calculate_area()
```

# Sample Program 2

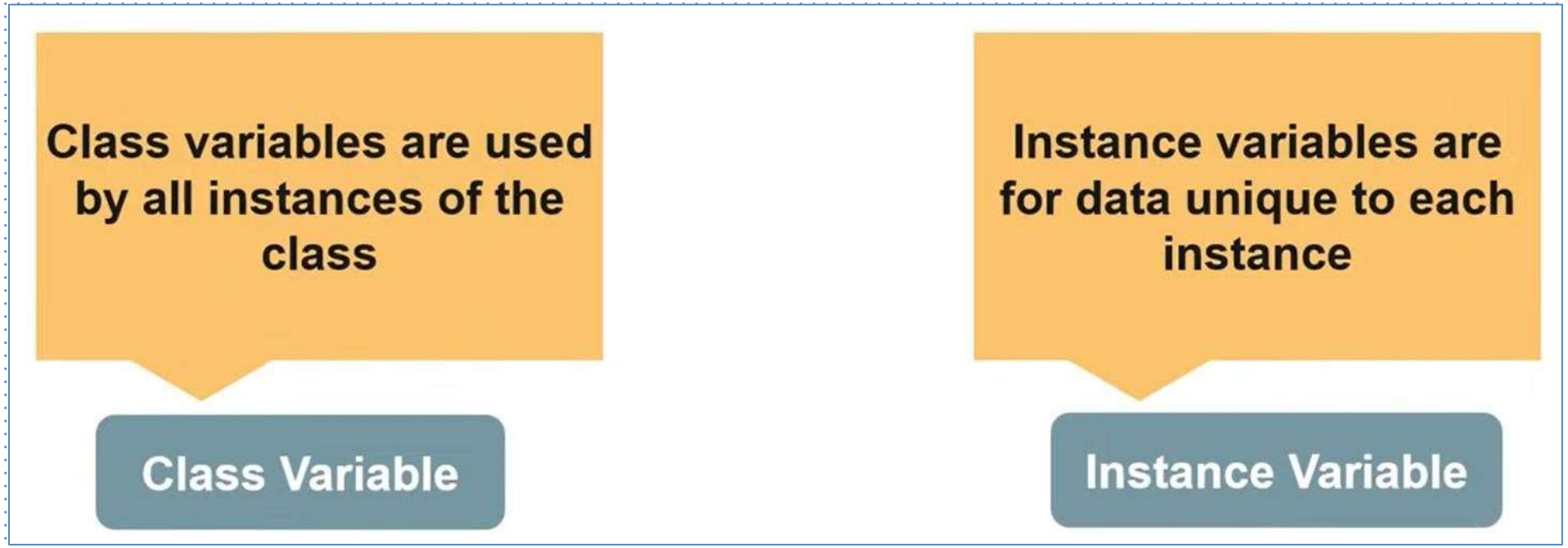
```
class Hotel:
    category = "Non-Veg"
    # Class attribute
    def __init__(self, loc, emp):
        self.location = loc          # Instance attribute
        self.employees = emp    # Instance attribute
    def returnloc(self):  # Instance method
        return f"{self.location}!"
    def increase_emp(self): Instance method
        self.employees += 1
    print(f"New Emp Joined at, {self.location}! now {self.employees} working")
```

# Sample Program 2 …

```
# Creating an instance of the class
    my_res = Hotel("Vellore", 20)       # Calling instance method
    print(my_res.returnloc())
    print(my_res.category)
    my_res.increase_emp()
    print(f"Total employees now: {my_res.employees}")
```

```
Vellore!
Non-Veg
New Emp Joined at, Vellore! now 21 working
Total employees now: 21
```

# Class/Static and Instance Variable



# Class Variable Vs Instance Variable

```
class Person:
    name = 'Raju'
    age = 21
    def show(cls):
        print(cls.name)
        print(cls.age)
        cls.age = 25
p1= Person()
print('Object p1')
p1.show()
p2 = Person()
p2.age = 23
print('Object p2')
p2.show()
print('Object p1')
p1.show()
print('Object p2')
p2.show()
```

```
class Person:
    def __init__(self):
        self.name = 'Raju'
        self.age = 21

    def show(self):
        print(self.name)
        print(self.age)

p1=  Person()
p2 =  Person()
p2.age = 23
print('Object p1')
p1.show()
print('Object p2')
p2.show()
print(id(p1))
print(id(p2))
```

# ABSTRACTION

# Abstraction

**Abstract Class** --> contain **Abstract Methods**

**Abstract Methods** --> ***Method without*** Definition.

**Concrete Class** --> ***Class without Abstract Methods***

***Object CAN'T be instantiated*** **for Abstract Class**

***Object CAN only be instantiated*** **for Concrete Class**

**Abstract Class** ===> **Concrete Class**

***A1()*** (***Inherited from Abstract Class***)

***A2()*** **Implement** or **Define** all the **Abstract Methods**, available in **Abstract Class**.

# DATA HIDING

# What is Data Hiding?

- **Data hiding** is a part of object-oriented programming, which is generally used to **hide the data information** from the user.
- It includes **internal object** details such as **data members**, **Internal working**.
- It maintained the **Data Integrity** and **restricted access** to the class member.
- We ***cannot directly access*** the **data** from **outside the class**.

# Working of Data Hiding

- We use double underscore ( __ ) *before the attribute name* and those attributes are **hide** from **other classes** in the program.

**Ex:**

**a=10 Public [within the program]**

**_a=10 Protected [Visible in Subclasses]**

**__a=10 Private [Visible within in the Class only]**

# Advantages of Data Hiding




- The **class objects** are disconnected from the **irrelevant data**.
- It enhances the **security** against hackers that are ***unable to access*** important data.
- It **isolates object** as the basic concept of **OOP**.
- It helps programmer from **incorrect linking** to the corrupt data.
- It provides the **high security** which stops ***damage to violate data by hiding it from the public***.

# Disadvantages of Data Hiding

- Sometimes programmers need to write the ***extra line of the code***.
- The ***data hiding*** **prevents** linkage that act as link between **visible** and **invisible data** makes the object faster.
- It forces the programmers to write extra code to ***hide the important data*** from the common users.

# ENCAPSULATION

# Encapsulation

- It describes the idea of **wrapping data** and the **methods** that work on data **within one unit**. (Combining Variables and Methods)
- Encapsulation is used to **restrict access** to **methods** and **variables**.
- The variables of a class will be ***hidden from other classes***, and can be accessed only through the methods of their current class.

# Encapsulation

```
class Employee:
    def __init__(self, name, project):
        self.name = name
        self.project = project

    def work(self):
        print(self.name, 'is working on', self.project)
```

Data Members

Method

Wrapping data and the methods that work on data within one unit

**Class** (Encapsulation)

# Python – Access Modifiers

- Let us see the **access modifiers** in Python to understand the concept of ***Encapsulation and data hiding*** –
  - **Public**
  - **Private**
  - **Protected**

# Python – Access Modifiers

**Public:** The public member is accessible from **inside** or **outside** the class.

**Private:** The private member is accessible only **inside class**. Define a private member by prefixing the member name with **two underscores**,

***For example:*** **__age**

**Protected:** From **inside the class and its sub-class**. Define a protected member by prefixing the member name with an **one underscore,**

***For example:*** **_points**

# Advantages – Encapsulation

- **Security**:

  **Encapsulation** protects an object from unauthorized access.

  It allows **private** and **protected** access levels to prevent accidental data modification.

- **Data Hiding**:

  The user would not be knowing what is going on behind the scene.

  They would only be knowing that to modify a data member, call the setter method. To read a data member, call the getter method.

- **Simplicity**:

  It simplifies the maintenance of the application by keeping classes separated and preventing them from tightly coupling with each other.

# MODULARITY

# What is Modularity ?

- **Modularity** refers to the act of partitioning the code into ***modules*** building them first followed by ***linking*** and ***finally*** combining them to form a **complete project**.
- **Modularity** ensures ***re-usability*** and thrives to minimize the duplication.
- A **module in Python** is nothing but a file containing Python ***definitions*** followed by ***methods*** & **statements**.
- The module name is generated out of the file name by removing the suffix ".py".

# Modularity

- **Modular programming** is the practice of **splitting our code** up into different files (modules) and then importing that code so that we can use it for many different projects.
- A module is typically any python file that contains only functions and classes

# Modularity

```python
# File name: myModule.py

def func(x):
    return x + 2

def func2(y):
    return y * 78
```

```python
# File name: main.py

import myModule

value = myModule.func(5)  # this will call the func function from myModule

print(value)  # this prints 7
```

# Modularity

- use the **import** keyword to **import modules** that are built into python or that we've created.

**Note:**

To import our own modules they must be in the same directory as the file we are importing from.

- To **avoid** the use of "**myModule**" we can import specific functions from our modules using a combination of keywords "**import**" and "**from**".

# Modularity

```python
# File name: myModule.py

def func(x):
    return x + 2

def func2(y):
    return y * 78
```

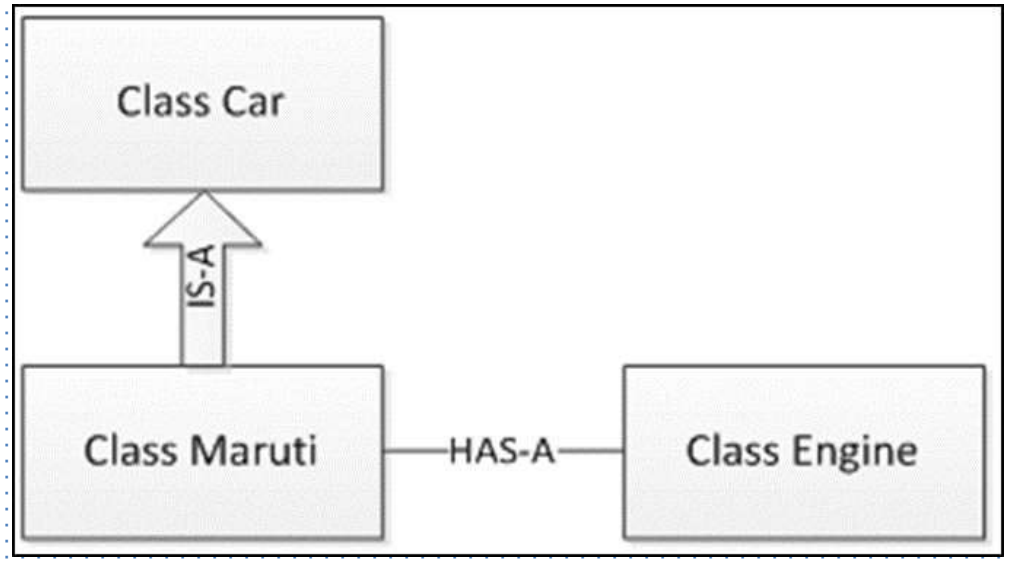
```python
# File name: main.py

from myModule import func

value = func(5)  # now we can just use func like a regular function

print(value)  # this prints 7
```

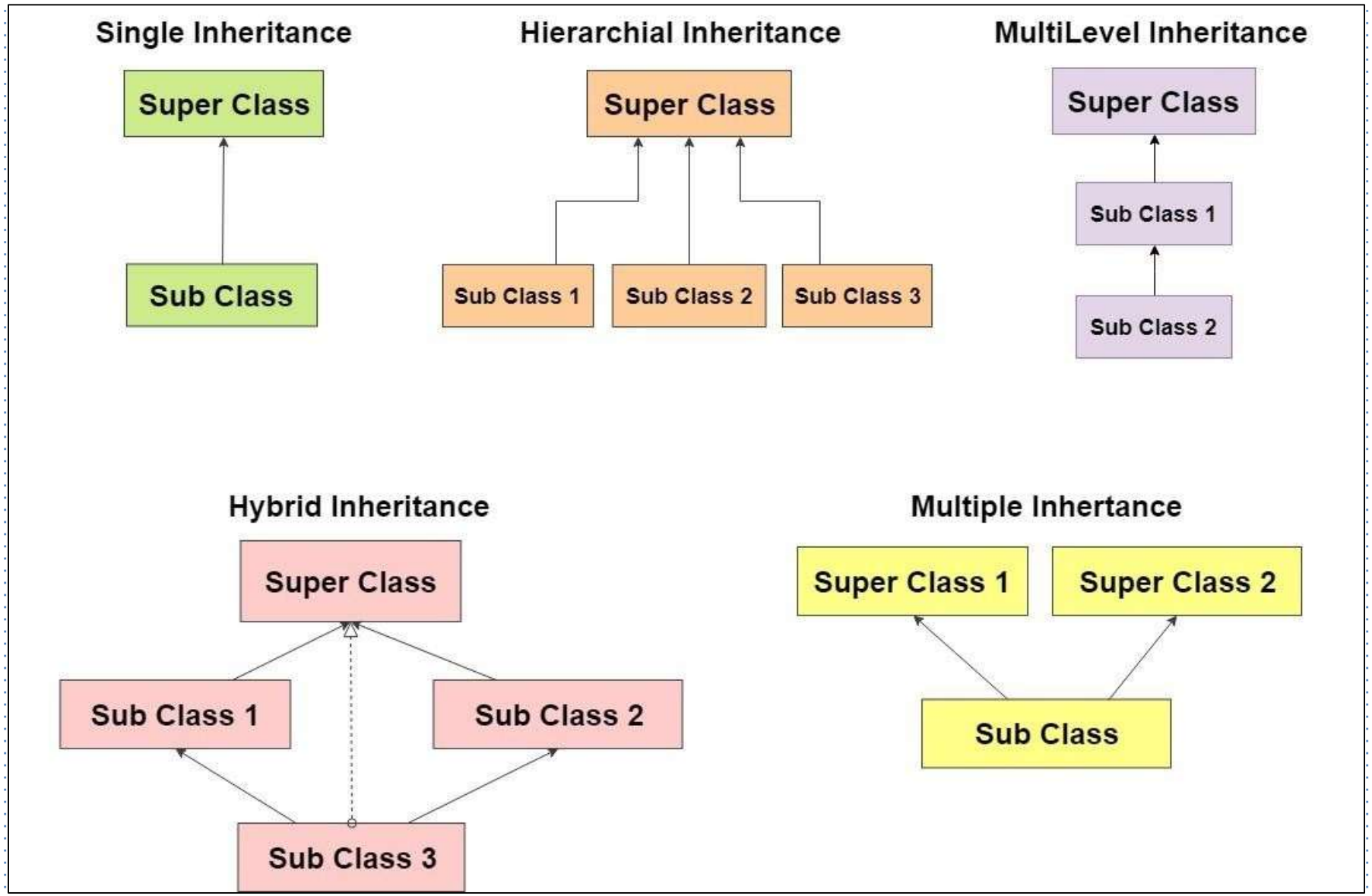
# INHERITANCE



# What is Inheritance?

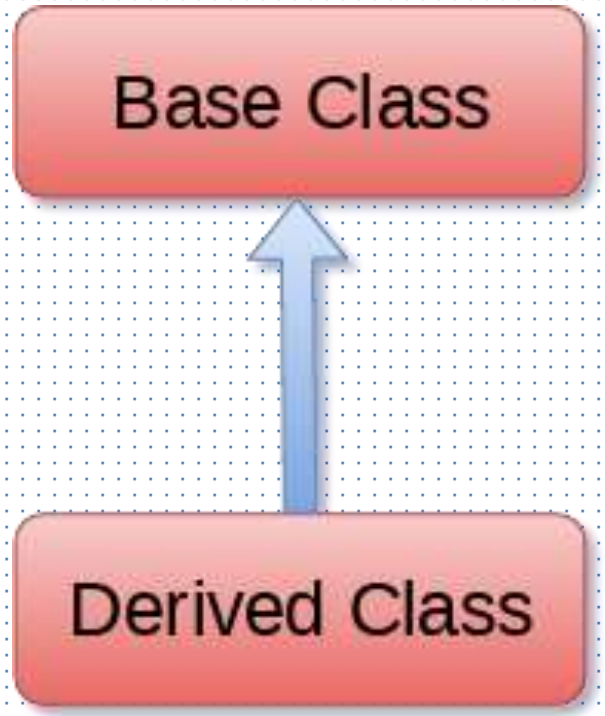



- **Inheritance** is an important aspect of the ***object-oriented paradigm***.
- **Inheritance** provides ***code reusability*** to the program because we can use an **existing class** to create a **new class** instead of creating it from start or scratch.
- **Parent class** is the class being **inherited** from, also called **base class**.
- **Child class** is the class that **inherits** from another class, also called **derived class**.


Single Inheritance
Super Class
Sub Class
Hierarchial Inheritance
Super Class
Sub Class 1
Sub Class 2
Sub Class 3
MultiLevel Inheritance
Super Class
Sub Class 1
Sub Class 2
Hybrid Inheritance
Super Class
Sub Class 1
Sub Class 2
Sub Class 3
Multiple Inhertance
Super Class 1
Super Class 2
Sub Class

# Inheritance

**Syntax** - To inherit a base class into the derived class.



# Different Types of Inheritance




- **Single inheritance:**

  When a **child class** inherits from only **one parent class**, it is called ***single inheritance***.

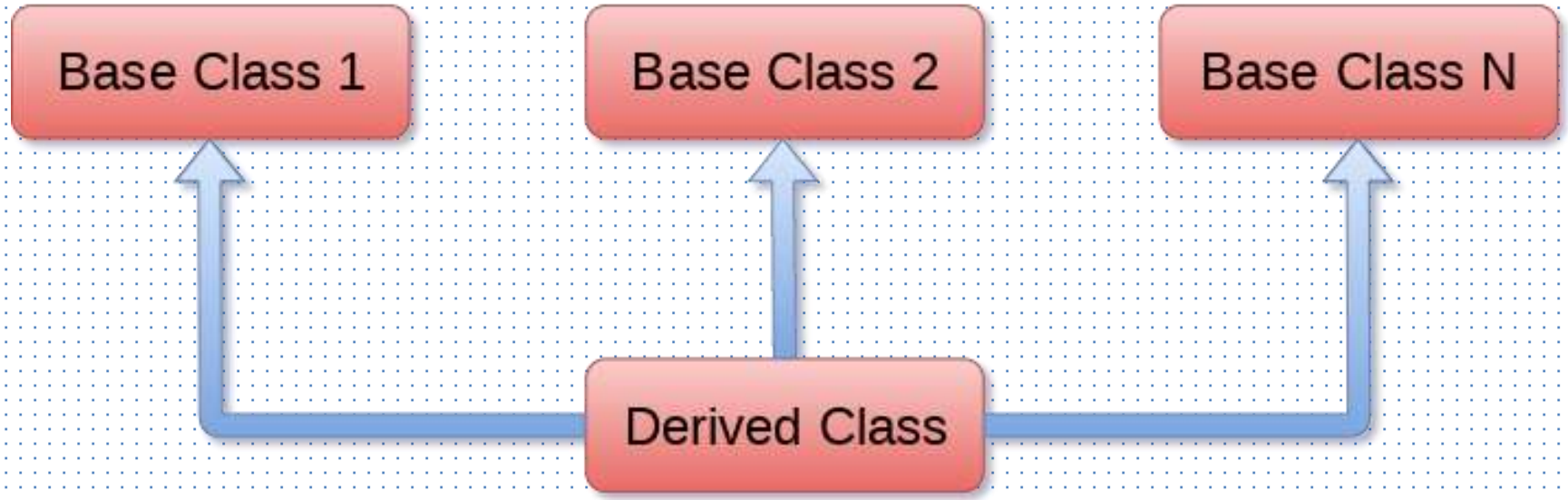
- **Multiple inheritances:**

  When a **child class** inherits from **multiple parent classes**, it is called ***multiple inheritances***.

- **Multilevel inheritance:**

  When we have a **child** and ***grandchild relationship***.

# Python Multiple inheritance



# Python Multiple inheritance

```
class Base1:
    <class-suite>

class Base2:
    <class-suite>
.
.
class BaseN:
    <class-suite>

class Derived(Base1, Base2, ..... BaseN):
    <class-suite>
```

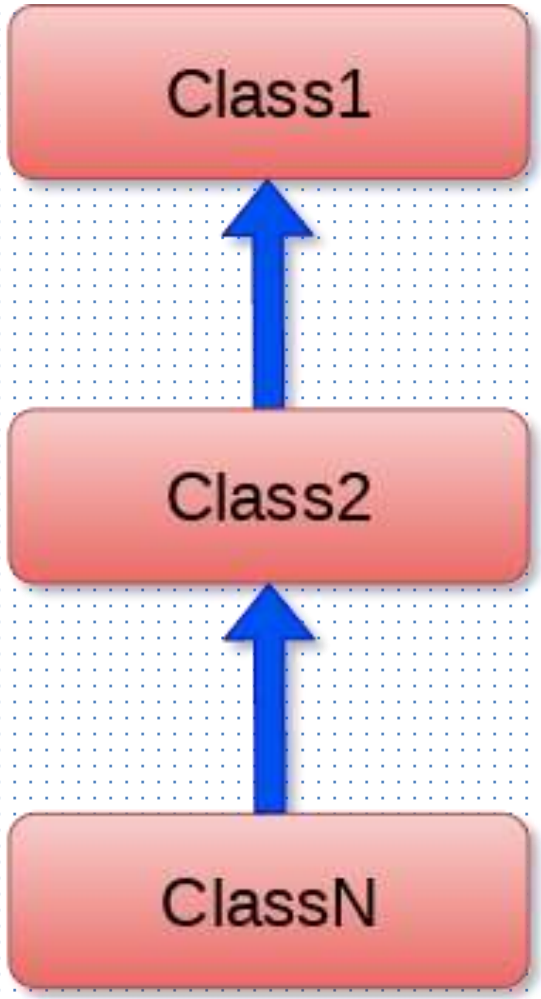
# Syntax- Python Multiple inheritance

```
class SuperClass1:
    # features of SuperClass1

class SuperClass2:
    # features of SuperClass2

class MultiDerived(SuperClass1, SuperClass2):
    # features of SuperClass1 + SuperClass2 + MultiDerived class
```

# Python Multi-Level inheritance



# Python Multi-Level inheritance

```
class class1:
    <class-suite>
class class2(class1):
    <class suite>
class class3(class2):
    <class suite>
.
.
```

# Multi-Level Inheritance

```
class SuperClass:
    # Super class code here

class DerivedClass1(BaseClass):
    # Derived class 1 code here

class DerivedClass2(DerivedClass1):
    # Derived class 2 code here
```

# Python Multi-Level inheritance

- Multi-Level inheritance is possible in python like other object-oriented languages.
- Multi-level inheritance is archived when a derived class inherits another derived class.
- There is no limit on the number of levels up to which, the multi-level inheritance is archived in python.

# Advantages of Inheritance

- It represents **real-world relationships** well.
- It provides the **reusability** of a code.
- It is **transitive in nature**, which means that if class B inherits from another class A, then all the subclasses of B would automatically inherit from class A.
- Inheritance offers a **simple**, **understandable** model structure.
- Less development and maintenance expenses result from an inheritance.

# INHERITANCE

















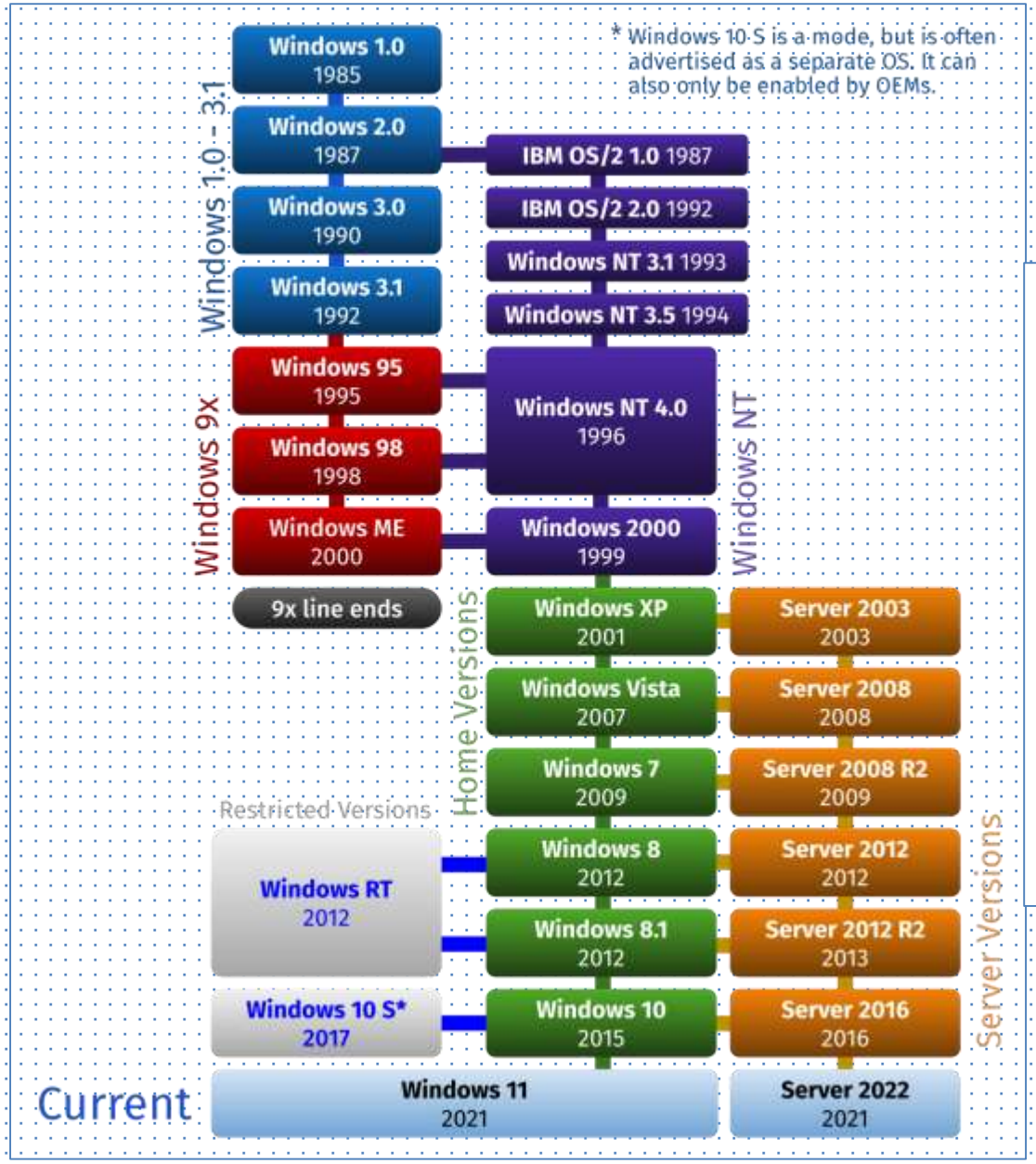


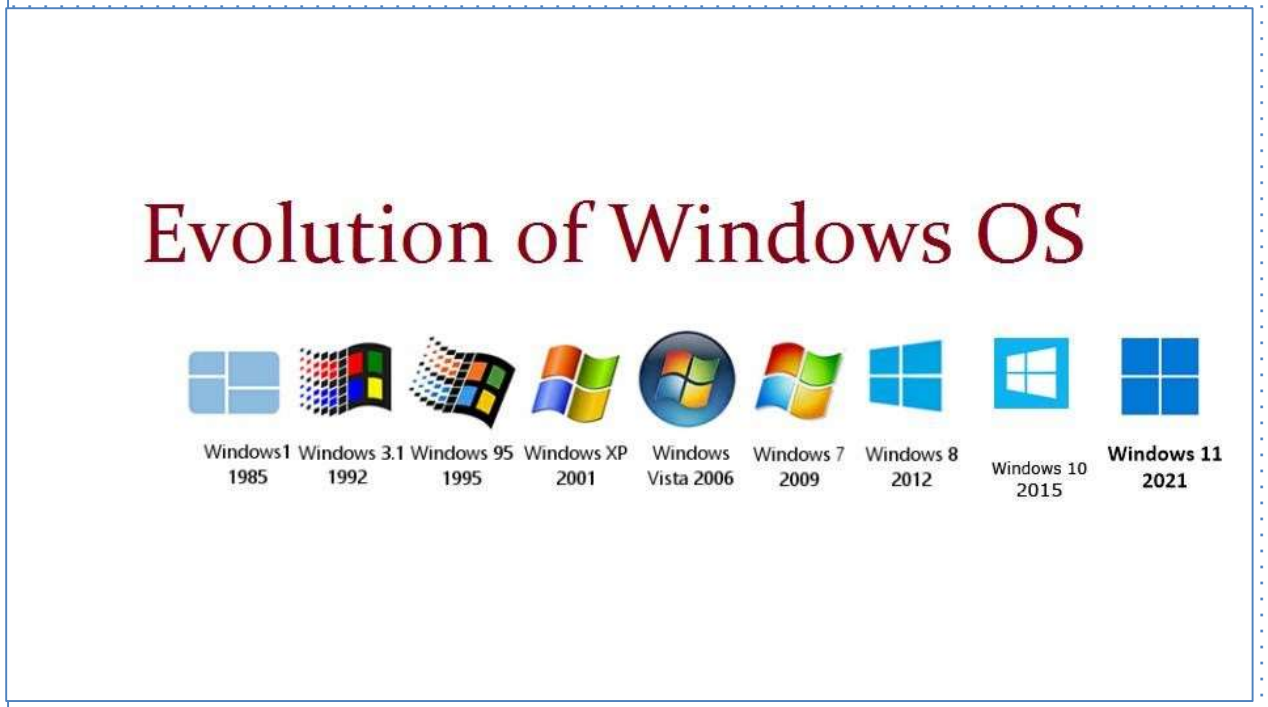




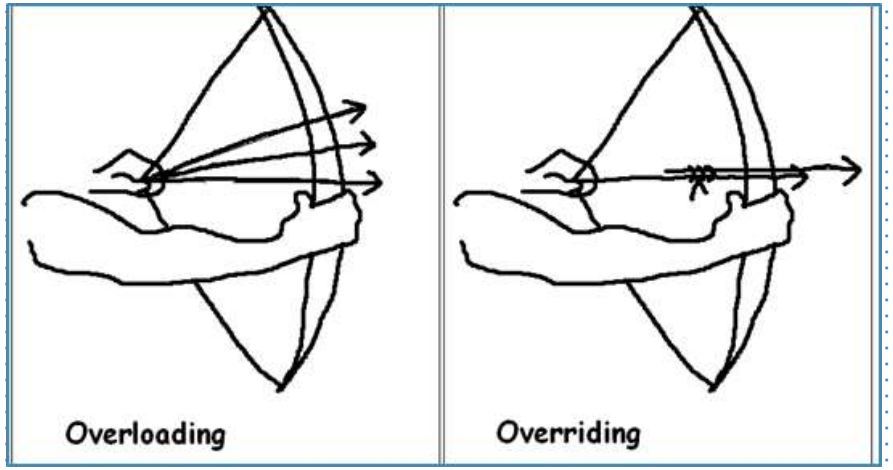


# INHERITANCE



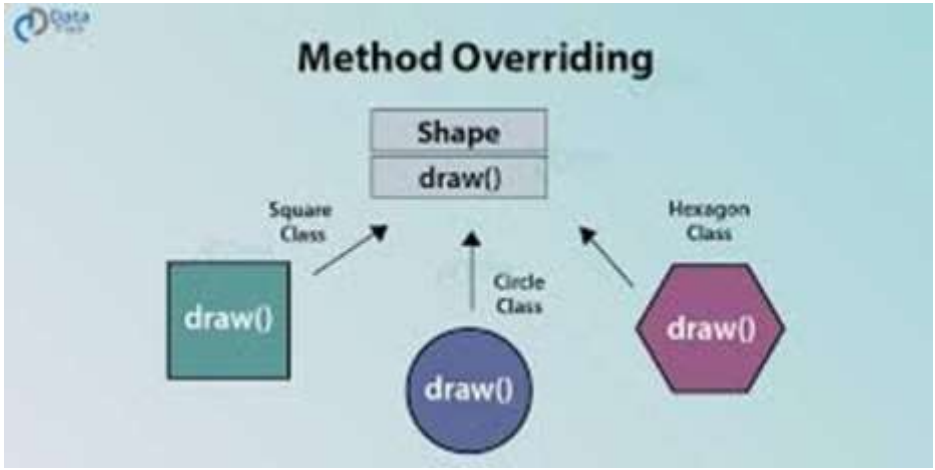
# Polymorphism



# What is Polymorphism?

- **Polymorphism** means ability to take various forms ***(Same Objet having Different Behaviour).***

  - ***Method Overloading***
  - ***Operator Overloading***
  - ***Method Overriding***



# Types in Polymorphism




✓ ***Method Overloading***:

A **Function** can be **called** with **Different Number of arguments**.

**NOTE**:

- Unlike C++ and Java., Python does **NOT** support Method Overloading.
- We **may overload** the **methods** **BUT** can only ***use the latest defined method***.

# Type in Polymorphism

- ***Method Overriding*:**

  **Multiple function** with **same name** and **same number of Parameters**

- ***Operator Overloading:***

  A Feature in Python that allows the **same operator** to have different meaning according to the context is called **operator overloading**.

***Ex: +***

***(i) Concatenation of String***

***(ii) Addition of Numbers***

# Method Overloading

**Advantages**:

- Overloading a method fosters **reusability**.

  For example, instead of writing multiple methods that differ only slightly, ***we can write one method and overload it***.

- Overloading also improves **code clarity** and eliminates **complexity**.

**Disadvantages**:

- Overloading can ***create confusion*** when used across inheritance boundaries. When used excessively, it becomes ***cumbersome to manage overloaded functions***.

# Advantages of Operator Overloading




- Improves **code readability** by allowing the use of familiar operators.
- Ensures that objects of a class behave consistently with built-in types and other user-defined types.
- Makes it **simpler to write code**, especially for **complex data types**.
- Allows for **code reuse** by implementing **one operator method** and using it for other operators.

# Advantages of Method Overriding

- Method overriding helps in writing a ***generic code*** based on the **parent class**.
- It provides ***multiple implementations*** of the **same method** and can be used to invoke **parent class overridden** methods using **super** keyword.
- It defines what behavior a class can have.

# Summary of the sessions …

- OOPs Concepts
- Class and Objects
- Variables and Methods in a Class
- Modularity
- Inheritance
- Polymorphism